Forecasting Average Traffic Speed on the PEMS08 Sensor Network

Single-horizon vs. multi-horizon graph neural networks

Vita Naglič Matevž Rom Nace Kovačič

Artificial Intelligence

Outline

Introduction

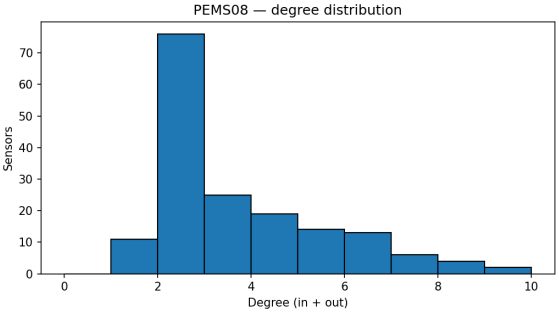
- A road network is covered by traffic sensors that measure speed every 5 minutes.
- **Goal:** given the recent past, forecast the average speed (mph) at each sensor for the near future.
- Why it matters: travel-time estimation, routing, congestion warnings, traffic management.
- It is a regression problem on a spatio-temporal graph:
  - **nodes** = sensors, **edges** = road connections,
  - each node carries a **time series** (speed, flow, occupancy).
- We build two neural networks:
  - **NN1** — predict speed exactly 60 minutes ahead;
  - **NN2** — predict the whole curve from +5 to +60 minutes.

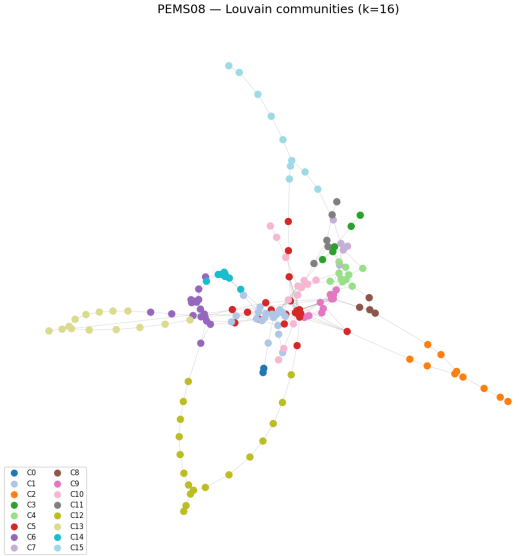
The PEMS dataset — focus on PEMS08

- **PeMS** = California "Performance Measurement System": loop detectors on freeways (PEMS03/04/07/08 are standard forecasting benchmarks).
- **PEMS08** (this work): district 8, Jul–Aug 2016.

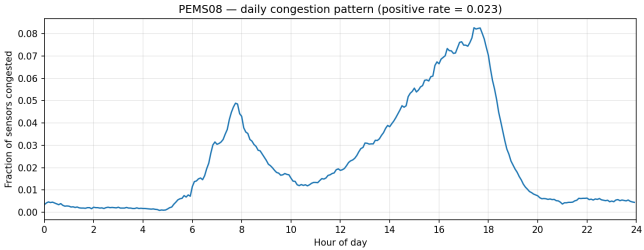
Property	Value
Sensors (nodes)	170
Time steps	17 856
Resolution	5 minutes
Duration	∼2 months
Channels	flow, occupancy, speed
Graph	directed, distance-weighted

The sensor graph — network structure





Channels & daily congestion



Fraction of sensors congested vs hour of day.

- Each sensor logs **flow**, **occupancy**, **speed** every 5 min — we forecast speed.
- Congestion = speed $<$ 60% of a sensor's free-flow (95th-percentile) speed: per-sensor & relative, not a fixed mph cutoff.
- Rare overall ($\sim$2.3%) but concentrated in AM ($\sim$8 h) and PM ($\sim$17 h) rush peaks; near zero overnight.
- Strong daily periodicity $\Rightarrow$ time-of-day features; congestion is the rare hard case.

NN1 — what goes in and what comes out

Task: for each sensor, predict its speed **one hour from now**.

- **Input (what the model sees):** the **last hour** of data from every sensor — 12 readings, one every 5 minutes.
- **Output (what it predicts):** one number per sensor — the speed in **60 minutes**.
- **Each reading gives 10 numbers per sensor:**
  - the 3 measured signals: flow, occupancy, speed;
  - the time of day and day of week (so it learns rush-hour patterns);
  - the average of its road-neighbours' signals (what nearby traffic is doing).
- We do **not** add "speed X minutes ago" features — the last hour is already in the input.

NN1 — split, normalisation, no leakage

- Temporal (chronological) split: 60% train / 20% val / 20% test.
  - train = past, test = future $\Rightarrow$ mimics real deployment.
  - not random k-fold: autocorrelation would leak the future and inflate scores.
- Per-sensor z-score normalisation using **training statistics only** (features and target); predictions denormalised back to mph for all metrics.
- No look-ahead: a gap of $W = 12$ steps is dropped at each split boundary, so an input window never crosses into another split.

Spatial (per time step) — graph convolution:

$$H = \mathrm{ReLU}(A\,X\,W_{gcn}), \quad A = \alpha\,A_{\text{fixed}} + (1-\alpha)\,\tilde{A}, \quad \tilde{A} = \mathrm{softmax}(\mathrm{ReLU}(E_1 E_2^{\top}))$$

- Adaptive adjacency $\tilde{A}$: a learned graph (captures correlations geometry misses).
- Residual $H = \mathrm{GCN}(X) + \mathrm{Linear}(X)$: keeps each sensor's own signal.
- **Temporal:** 2-layer GRU over the 12 steps (shared across sensors).
- **Head:** $\mathrm{Linear}(\text{hidden} \to 1) \Rightarrow$ one speed (60 min). No sigmoid.
- **Training:** Huber loss ($\delta = 1$), Adam (lr 10^{-3}), early stopping on val MAE, seed 42, $\sim$56k params, trained on Apple-Silicon GPU (MPS).

NN2 — predicting the whole interval [+5, +60] min

Task: for every sensor, predict speed at all 12 steps 5, 10, . . . , 60 min ahead.

- "Any segment at any near-future time" — the data lives on a 5-min grid.
- **Target** grows from [170] to [170, 12] (one value per horizon).
- Same 10 features, same 12-step input window as NN1.
- Standard benchmark setup (DCRNN, Graph WaveNet report 15/30/60 min jointly).

Both models are kept in the code (flag `--single-horizon` selects NN1); NN2 is the default. Neither overwrites the other.

NN2 — what changes vs NN1

- **Same** spatial GCN + adaptive adjacency + residual, **same** GRU, **same** split, normalisation, loss, training recipe.
- **Only the output head changes:**

$$\mathrm{Linear}(\text{hidden} \to 1) \implies \mathrm{Linear}(\text{hidden} \to 12)$$

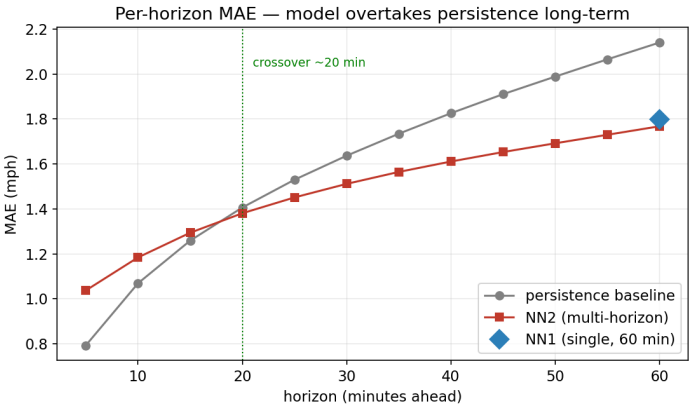
- Output shape per sample: $[N] \Rightarrow [N, 12]$; Huber loss summed over all 12 horizons.
- Predicting all horizons jointly acts as multi-task regularisation (one model, full curve).
- Cost: $\sim$60k params (vs $\sim$56k) — essentially free.

Results — NN1 vs NN2 vs baselines

Model (test set)	MAE	RMSE	MAPE
Persistence (avg over 5–60 min)	1.6133	3.7811	3.08%
Persistence (60 min only)	2.1403	4.9692	4.21%
NN1 — single-horizon (60 min)	1.7979	4.4583	4.13%
NN2 — multi-horizon (60 min)	**1.7677**	4.2927	3.98%
NN2 — multi-horizon (avg)	**1.4897**	3.6037	3.23%

- At 60 min, NN2 matches/beats NN1 while *also* covering all shorter horizons.
- NN2 beats the average persistence by +7.7% — and that average is hard (see next slide).

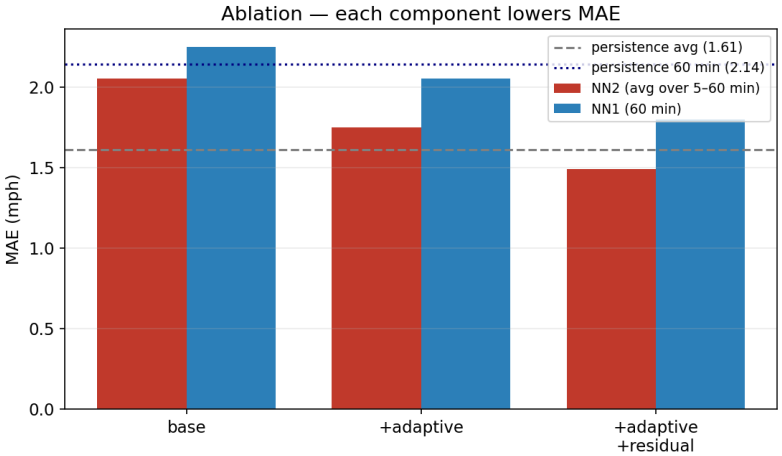
Results — the crossover (per-horizon MAE)



Key finding

- Persistence is near-perfect short-term (speed barely changes in 5 min).
- The model overtakes at $\sim$20 min and the gap grows: $+17.4\%$ at 60 min.
- $\Rightarrow$ judge models per horizon, not by a single average.

Results — ablation (each component helps)



- Fixed-graph **base** is weakest; + adaptive adjacency and + residual each lower MAE.
- Same trend for NN1 and NN2 — the gains come from architecture, not scale.

Conclusions

- Two graph neural networks forecast PEMS08 speed: NN1 (60 min) and NN2 (full $+5 \ldots +60$ min curve).
- GCN + learned adjacency + residual + GRU beats strong baselines at the horizons that matter.
- NN2 gives the whole forecast curve from one model, matching NN1 at 60 min — the more useful, practical choice.
- Crossover insight: short-term is owned by persistence; learned models win long-term.
- Future work: bidirectional diffusion, node-specific weights, temporal attention, congestion-weighted loss.